

Libpcap - API

Presenter:

Assist.Prof.Dr. Fatih ABUT

Agenda

- Installing libpcap
- Some C stuff
- Basic libpcap program
 - Grab a device to sniff
 - Filters/Event Loops
 - Packet structure

Install on Linux

- `gunzip libpcap-0.7.1.tar.gz`
- `tar -xvf libpcap-0.7.1.tar`
- `cd libpcap-0.7.1`
- `./configure`
- `make`

- Or `apt-get install libpcap-dev`

Avoiding C Gotchas



- Always declare variables at the beginning of a block (no Java/C++ messiness!!)
- Nothing ‘new’: Always free what you malloc
malloc(sizeof (thingYouWantToAllocate));
- Always check the return value (no Exceptions!)

```
if (thing_didnt_work()) {  
    fprintf(stderr, "ERROR: thing didn't work\n");  
    exit(-1);  
} /* if (thing_didnt_work) */
```

C cont'd

- Output is formatted.

```
char person[ ] = "baby";
```

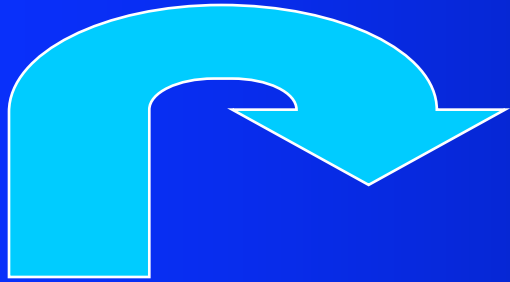
```
printf("give me %d, %s\n", 5, person);
```

%d: int

%x: hex

%s: string

%f: double



Get to the point!

- Pass by reference explicitly

- Pass-by-reference prototype

```
int doSomething( Thing *);
```

Choice 1:

```
Thing *t;
```

```
doSomething( t );
```

Choice 2:

```
Thing t;
```

```
doSomething( &t );
```

- Arrays are always in reference mode:

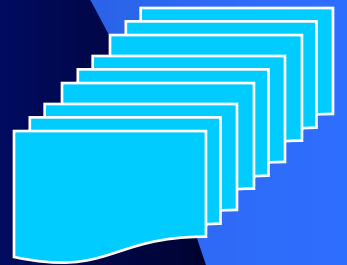
char * is like char[0]

Finally...

- C is NOT an object-oriented language

Most frequent data structure is a struct.
Under the covers this is an array of contiguous bytes.

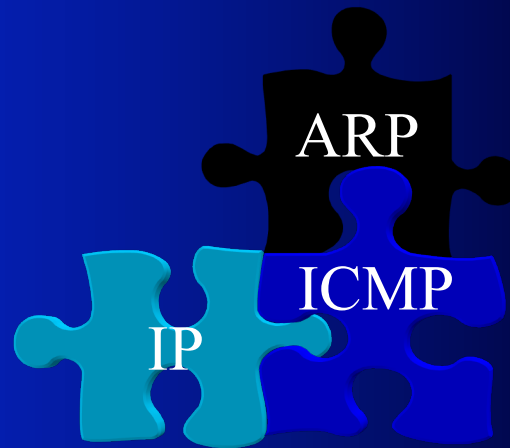
```
struct pcap_pkthdr {  
    struct timeval ts; //time stamp  
    bpf_u_int32 caplen; // length of  
                        //portion present  
    bpf_u_int32 len; //packet length  
}
```



Overview of libpcap



- What to include and how to compile
- Going Live
- Main Event Loop
- Reading from a packet
- Filters



What to include and how to compile

- gcc sniff.c -lpcap -o sniff
- You must be root or admin
- Some headers I've used.

```
#include <pcap.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <sys/socket.h>
```

```
#include <netinet/if_ether.h>
```

```
#include <netinet/in.h>
```

```
#include <netinet/ip.h>
```

```
#include <netinet/tcp.h>
```

```
#include <arpa/inet.h>
```

For Windows:

```
#include <winsock.h>
```

Getting onto the NIC

```
int main(int argc, char **argv) {
    char *dev;                /* name of the device to use */
    pcap_t* descr;            /* pointer to device descriptor */
    struct pcap_pkthdr hdr;    /* struct: packet header */
    const u_char *packet;      /* pointer to packet */
    bpf_u_int32 maskp;         /* subnet mask */
    bpf_u_int32 netp;          /* ip */
    char errbuf[PCAP_ERRBUF_SIZE];

    /* ask pcap to find a valid device to sniff */
    dev = pcap_lookupdev(errbuf);
    if(dev == NULL)
        { printf("%s\n",errbuf); exit(1); }
    printf("DEV: %s\n",dev);
```

Going Live!



```
/* ask pcap for the network address and mask of the device */  
pcap_lookupnet(dev,&netp,&maskp,errbuf);
```

```
descr = pcap_open_live(dev,BUFSIZ, 0, -1,errbuf);
```

```
/* BUFSIZ is max packet size to capture, 0 is promiscuous, -1  
means don't wait for read to time out. */
```

```
if(descr == NULL)  
{  
    printf("pcap_open_live(): %s\n",errbuf);  
    exit(1);  
}
```

Once live, capture a packet.

```
packet = pcap_next(descr, &hdr);  
if (packet == NULL) {  
    printf("It got away!\n");  
    exit(1);  
}  
else printf("one lonely packet.\n");  
return 0;  
} //end main
```

Main Event Loop

```
void my_callback(u_char *useless, const struct
    pcap_pkthdr* pkthdr, const u_char* packet) {
    //do stuff here with packet
}
```

```
int main(int argc, char **argv) {
    //open and go live
```

```
    pcap_loop(descr, -1, my_callback, NULL);
    return 0;
}
```

What is an ethernet header?

From `#include<netinet/if_ether.h>`

```
struct ether_header {  
    u_int8_t ether_dhost[ETH_ALEN]; /* 6 bytes destination */  
    u_int8_t ether_shost[ETH_ALEN]; /* 6 bytes source addr */  
    u_int16_t ether_type;             /* 2 bytes ID type */  
} __attribute__((__packed__));
```

Some ID types:

```
#define ETHERTYPE_IP    0x0800 /* IP */  
#define ETHERTYPE_ARP  0x0806 /* Address resolution */
```

Is this platform independent?

NO!

So we may need to swap bytes to read the data.

```
struct ether_header *eptr;    /* where does this go? */
```

```
eptr = (struct ether_header *) packet;
```

```
/* Do a couple of checks to see what packet type we have..*/
```

```
if (ntohs (eptr->ether_type) == ETHERTYPE_IP) {  
    printf("Ethernet type hex:%x dec:%d is an IP packet\n",  
        ntohs(eptr->ether_type), ntohs(eptr->ether_type));
```

```
} else if (ntohs (eptr->ether_type) == ETHERTYPE_ARP) {  
    printf("Ethernet type hex:%x dec:%d is an ARP packet\n",  
        ntohs(eptr->ether_type), ntohs(eptr->ether_type));  
}
```

Filter – we don't need to see every packet!

- Filters are strings. They get “compiled” into “programs”

struct bpf_program fp; //where does it go?

- Just before the event loop:
if (**pcap_compile(descr,&fp,argv[1],0,netp) == -1**)
{ fprintf(stderr,"Error calling pcap_compile\n");
 exit(1);
}

if (**pcap_setfilter(descr,&fp) == -1**) {
 fprintf(stderr,"Error setting filter\n");
 exit(1);
}

Some typical filters

```
./sniff "dst port 80"
```

```
./sniff "src host 128.226.121.120"
```

```
./sniff "less 50"
```

(grab all packets less than 50 bytes)

```
./sniff "ip proto \udp"
```

(must use the escape character, \, for protocol names)

```

34  #include <stdio.h>
35  #include <pcap.h>
36  #include <stddef.h>
37
38  void got_packet(u_char *args, const struct pcap_pkthdr *header, const u_char *packet)
39  {
40      return;
41  }
42
43  int main(int argc, char *argv[])
44  {
45      pcap_t *ph; //pcap_handler
46      char *dev;
47      char errbuf[PCAP_ERRBUF_SIZE];
48
49      if (argc != 2) {
50          printf ("Usage: %s DEVICE\n",argv[0]);
51          return 1;
52      }
53
54      dev = argv[1];
55
56      struct bpf_program fp;          /* The compiled filter expression */
57
58      /* The filter expression */ //only Segments with SYN, SYN+ACK or ACK Flags
59      char filter_exp[] = "(tcp[13] == 2 || tcp[13] == 18 ||
60                          tcp[13] == 16) && (port 8080 || port 80)";
61
62      bpf_u_int32 mask;                /* The netmask of our sniffing device */
63      bpf_u_int32 net;                /* The IP of our sniffing device */
64      int num_packets = -1;           /* Number of packets to capture;
65                                      set to -1 to capture as long as an error occur*/
66
67      printf("Device: %s\n", dev);
68      printf("Number of packets to capture: %d\n", num_packets);
69      printf("Filter expression: %s\n", filter_exp);

```

```

71  /* Find the properties for the device */
72  if (pcap_lookupnet(dev, &net, &mask, errbuf) == -1)
73  {
74      fprintf(stderr, "Can't get netmask for device %s\n", dev);
75      net = 0;
76      mask = 0;
77  }
78
79  /* Open the session in promiscuous mode */
80  ph=pcap_open_live(dev, BUFSIZ, 0,1000, errbuf);
81  if (ph == NULL) {
82      fprintf(stderr, "Couldn't open device %s: %s\n", dev, errbuf);
83      return EXIT_FAILURE;
84  }
85
86  /* Compile the filter */
87  if (pcap_compile(ph, &fp, filter_exp, 0, net) == -1)
88  {
89      fprintf(stderr, "Couldn't parse filter %s: %s\n",
90              filter_exp, pcap_geterr(ph));
91      return EXIT_FAILURE;
92  }
93  /* Apply the filter */
94  if (pcap_setfilter(ph, &fp) == -1)
95  {
96      fprintf(stderr, "Couldn't install filter %s: %s\n",
97              filter_exp, pcap_geterr(ph));
98      return EXIT_FAILURE;
99  }
100
101  /* Capture packets */
102  printf ("Starting capture...\n");
103  pcap_loop(ph, num_packets, got_packet, NULL);
104
105  /* Close the session */
106  pcap_freecode(&fp);
107  pcap_close(ph);
108
109  printf("\nCapture complete.\n");
110
111  return(0);
112 }

```